

8. Generics (continued)

Multiple Type Parameters:

java



```
// Class with multiple type parameters
public class Pair<K, V> {
    private K key;
    private V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() {
        return key;
    }

    public V getValue() {
        return value;
    }
}

// Using multiple type parameters
Pair<String, Integer> person = new Pair<>("John", 25);
String name = person.getKey();
Integer age = person.getValue();
```

Bounded Type Parameters:

java



```
// Type parameter bounded to Number or its subclasses
public class MathBox<T extends Number> {
    private T value;

    public MathBox(T value) {
        this.value = value;
    }

    public double sqrt() {
        return Math.sqrt(value.doubleValue());
    }
}

// Valid usage
MathBox<Integer> intBox = new MathBox<>(16);
double result = intBox.sqrt(); // 4.0

// Invalid usage (String is not a Number)
// MathBox<String> stringBox = new MathBox<>("Hello"); // Compile error
```

Wildcard Types:

java



```
// Unknown wildcard
public void printList(List<?> list) {
    for (Object item : list) {
        System.out.println(item);
    }
}

// Upper bounded wildcard
public double sumOfList(List<? extends Number> list) {
    double sum = 0;
    for (Number n : list) {
        sum += n.doubleValue();
    }
    return sum;
}

// Lower bounded wildcard
public void addNumbers(List<? super Integer> list) {
    list.add(1);
    list.add(2);
    // list.add("String"); // Compile error
}
```

Type Erasure:

Java generics use type erasure at runtime, which means generic type information is removed during compilation. The following code demonstrates this concept:



```
public void showErasure() {  
    List<String> stringList = new ArrayList<>();  
    List<Integer> intList = new ArrayList<>();  
  
    // At runtime, both have the same class  
    System.out.println(stringList.getClass() == intList.getClass()); // true  
  
    // Can't do: if (list instanceof List<String>)  
    // Can do: if (list instanceof List)  
}
```

9. Lambda Expressions and Functional Interfaces

Functional Interfaces:

A functional interface has exactly one abstract method.

java



```
// Predefined functional interfaces
@FunctionalInterface
interface Runnable {
    void run();
}

@FunctionalInterface
interface Callable<V> {
    V call() throws Exception;
}

// Custom functional interface
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}
```

Lambda Expressions:

java



```
// Traditional anonymous class
Runnable oldWay = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hello World");
    }
};

// Lambda expression
Runnable lambdaWay = () -> System.out.println("Hello World");

// Lambda with parameters
Calculator addition = (a, b) -> a + b;
Calculator subtraction = (a, b) -> a - b;

// Using the lambda
int result1 = addition.calculate(5, 3);    // 8
int result2 = subtraction.calculate(5, 3); // 2

// Lambda with multiple statements
Calculator complexOp = (a, b) -> {
    int result = a * b;
    return result + a + b;
};
```

Common Functional Interfaces in java.util.function:

java



```
import java.util.function.*;

// Predicate: takes one argument, returns boolean
Predicate<String> isEmpty = s -> s.isEmpty();
boolean result = isEmpty.test(""); // true

// Function: takes one argument, returns result of any type
Function<String, Integer> length = s -> s.length();
int len = length.apply("Hello"); // 5

// Consumer: takes one argument, returns nothing
Consumer<String> printer = s -> System.out.println(s);
printer.accept("Hello World"); // Prints "Hello World"

// Supplier: takes no arguments, returns result
Supplier<Double> random = () -> Math.random();
double value = random.get(); // Random value between 0 and 1

// BiFunction: takes two arguments, returns result
BiFunction<Integer, Integer, Integer> multiply = (a, b) -> a * b;
int product = multiply.apply(5, 3); // 15

// UnaryOperator: takes one argument, returns same type
UnaryOperator<String> toUpper = s -> s.toUpperCase();
String upper = toUpper.apply("hello"); // "HELLO"

// BinaryOperator: takes two arguments of same type, returns same type
BinaryOperator<Integer> max = (a, b) -> a > b ? a : b;
int maxValue = max.apply(10, 5); // 10
```

Method References:

java



```
import java.util.Arrays;
import java.util.List;

List<String> names = Arrays.asList("John", "Alice", "Bob");

// Method reference to static method
names.forEach(System.out::println);

// Method reference to instance method
String prefix = "User: ";
names.forEach(prefix::concat);

// Method reference to constructor
List<Integer> lengths = names.stream()
    .map(String::length)
    .toList();
```

10. Stream API

Creating Streams:

java



```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Stream;
import java.util.stream.IntStream;

// From collection
List<String> list = Arrays.asList("a", "b", "c");
Stream<String> streamFromList = list.stream();

// From array
String[] array = {"a", "b", "c"};
Stream<String> streamFromArray = Arrays.stream(array);

// Stream.of
Stream<String> streamOf = Stream.of("a", "b", "c");

// Empty stream
Stream<String> emptyStream = Stream.empty();

// Infinite streams
Stream<Integer> infiniteStream = Stream.iterate(0, n -> n + 1);
Stream<Double> randomStream = Stream.generate(Math::random);

// Primitive streams
IntStream intStream = IntStream.range(1, 5); // 1, 2, 3, 4
IntStream closedRange = IntStream.rangeClosed(1, 5); // 1, 2, 3, 4, 5
```

Intermediate Operations:

java



```
List<String> names = Arrays.asList("John", "Alice", "Bob", "Charlie", "Alex");

// Filter
Stream<String> filtered = names.stream()
    .filter(name -> name.startsWith("A"));

// Map
Stream<Integer> lengths = names.stream()
    .map(String::length);

// FlatMap
List<List<Integer>> nestedLists = Arrays.asList(
    Arrays.asList(1, 2),
    Arrays.asList(3, 4)
);
Stream<Integer> flattened = nestedLists.stream()
    .flatMap(list -> list.stream());

// Distinct
Stream<String> distinct = names.stream()
    .distinct();

// Sorted
Stream<String> sorted = names.stream()
    .sorted();
Stream<String> customSorted = names.stream()
    .sorted((a, b) -> b.compareTo(a)); // Reverse order

// Peek (for debugging)
Stream<String> peeked = names.stream()
    .peek(name -> System.out.println("Processing: " + name));
```

```
// Limit
Stream<String> limited = names.stream()
    .limit(3); // First 3 elements

// Skip
Stream<String> skipped = names.stream()
    .skip(2); // Skip first 2 elements
```

Terminal Operations:

java



```
// forEach
names.stream()
    .forEach(System.out::println);

// collect
import java.util.stream.Collectors;

List<String> collectedList = names.stream()
    .collect(Collectors.toList());

String joined = names.stream()
    .collect(Collectors.joining(", "));

// reduce
Optional<String> reduced = names.stream()
    .reduce((a, b) -> a + ", " + b);

// find operations
Optional<String> any = names.stream()
    .findAny();
Optional<String> first = names.stream()
    .findFirst();

// match operations
boolean allMatch = names.stream()
    .allMatch(name -> name.length() > 0);
boolean anyMatch = names.stream()
    .anyMatch(name -> name.startsWith("A"));
boolean noneMatch = names.stream()
    .noneMatch(name -> name.length() > 10);

// count
```

```
long count = names.stream()
    .count();

// min/max
Optional<String> min = names.stream()
    .min((a, b) -> a.length() - b.length());
Optional<String> max = names.stream()
    .max(Comparator.naturalOrder());

// toArray
String[] array = names.stream()
    .toArray(String[]::new);
```

Common Stream Operations:

java




```
List<Person> people = Arrays.asList(
    new Person("John", 25),
    new Person("Alice", 30),
    new Person("Bob", 20),
    new Person("Charlie", 35)
);

// Filter and collect
List<Person> adults = people.stream()
    .filter(p -> p.getAge() >= 18)
    .collect(Collectors.toList());

// Group by
Map<Integer, List<Person>> byAge = people.stream()
    .collect(Collectors.groupingBy(Person::getAge));

// Average
double averageAge = people.stream()
    .mapToInt(Person::getAge)
    .average()
    .orElse(0);

// Statistics
IntSummaryStatistics stats = people.stream()
    .mapToInt(Person::getAge)
    .summaryStatistics();
// stats.getAverage(), stats.getMax(), stats.getMin(), stats.getSum(), stats.getCo

// Partitioning
Map<Boolean, List<Person>> partitioned = people.stream()
    .collect(Collectors.partitioningBy(p -> p.getAge() > 30));
```

```
// Converting to map  
Map<String, Integer> nameToAge = people.stream()  
    .collect(Collectors.toMap(Person::getName, Person::getAge));
```

11. File I/O and Serialization

File I/O

Working with Files:

java



```
import java.io.File;
import java.io.IOException;

// Creating a File object
File file = new File("example.txt");

// File information
boolean exists = file.exists();
boolean isFile = file.isFile();
boolean isDirectory = file.isDirectory();
long length = file.length(); // Size in bytes
String name = file.getName();
String path = file.getPath();
String absolutePath = file.getAbsolutePath();

// File operations
boolean created = file.createNewFile(); // Create new file
boolean deleted = file.delete();        // Delete file
boolean renamed = file.renameTo(new File("newname.txt")); // Rename

// Directory operations
File dir = new File("mydir");
boolean dirCreated = dir.mkdir();        // Create directory
File[] contents = dir.listFiles();       // List files in directory
```

Traditional I/O (byte streams):

java



```
import java.io.*;

// Writing bytes
try (FileOutputStream fos = new FileOutputStream("file.bin");
    BufferedOutputStream bos = new BufferedOutputStream(fos)) {

    byte[] data = {65, 66, 67, 68}; // ABCD
    bos.write(data);

} catch (IOException e) {
    e.printStackTrace();
}

// Reading bytes
try (FileInputStream fis = new FileInputStream("file.bin");
    BufferedInputStream bis = new BufferedInputStream(fis)) {

    int byteRead;
    while ((byteRead = bis.read()) != -1) {
        System.out.print((char) byteRead); // Convert byte to char
    }

} catch (IOException e) {
    e.printStackTrace();
}
```

Character Streams:

java



```
// Writing text
try (FileWriter writer = new FileWriter("file.txt");
    BufferedWriter buffered = new BufferedWriter(writer)) {

    buffered.write("Hello, World!");
    buffered.newLine(); // Add new line
    buffered.write("This is a text file.");

} catch (IOException e) {
    e.printStackTrace();
}

// Reading text
try (FileReader reader = new FileReader("file.txt");
    BufferedReader buffered = new BufferedReader(reader)) {

    String line;
    while ((line = buffered.readLine()) != null) {
        System.out.println(line);
    }

} catch (IOException e) {
    e.printStackTrace();
}
```

NIO for Modern File I/O (Java 7+):

java



```
import java.nio.file.*;
import java.nio.charset.StandardCharsets;
import java.util.List;

// Writing files
Path path = Paths.get("file.txt");
List<String> lines = Arrays.asList("Line 1", "Line 2", "Line 3");

try {
    Files.write(path, lines, StandardCharsets.UTF_8);
} catch (IOException e) {
    e.printStackTrace();
}

// Reading files
try {
    List<String> readLines = Files.readAllLines(path, StandardCharsets.UTF_8);
    for (String line : readLines) {
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}

// File operations
try {
    // Copy file
    Path source = Paths.get("source.txt");
    Path target = Paths.get("target.txt");
    Files.copy(source, target, StandardCopyOption.REPLACE_EXISTING);

    // Move/rename file
```

```
Files.move(source, Paths.get("renamed.txt"), StandardCopyOption.REPLACE_EXISTING);

// Delete file
Files.delete(Paths.get("toDelete.txt"));

// Create directories
Files.createDirectories(Paths.get("path/to/directory"));
} catch (IOException e) {
    e.printStackTrace();
}
```

Serialization

Serializing Objects:

java



```

import java.io.*;

// Serializable class
class Person implements Serializable {
    // Serialization version UID
    private static final long serialVersionUID = 1L;

    private String name;
    private int age;
    private transient String tempData; // Won't be serialized

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
        this.tempData = "Temporary";
    }

    @Override
    public String toString() {
        return "Person{name='" + name + "', age=" + age +
            ", tempData='" + tempData + "'}";
    }
}

// Serializing object
try (FileOutputStream fos = new FileOutputStream("person.ser");
    ObjectOutputStream oos = new ObjectOutputStream(fos)) {

    Person person = new Person("John", 30);
    oos.writeObject(person);
    System.out.println("Object serialized");
}

```

```
} catch (IOException e) {  
    e.printStackTrace();  
}  
  
// Deserializing object  
try (FileInputStream fis = new FileInputStream("person.ser");  
    ObjectInputStream ois = new ObjectInputStream(fis)) {  
  
    Person person = (Person) ois.readObject();  
    System.out.println("Object deserialized: " + person);  
    // Note: transient fields will be null or default values  
  
} catch (IOException | ClassNotFoundException e) {  
    e.printStackTrace();  
}
```

Custom Serialization:

java



```

class SpecialPerson implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name;
    private transient String sensitiveData;

    // Custom serialization
    private void writeObject(ObjectOutputStream out) throws IOException {
        out.defaultWriteObject(); // Default serialization for non-transient fields

        // Custom serialization for sensitive data (e.g., encrypt)
        out.writeObject(encrypt(sensitiveData));
    }

    // Custom deserialization
    private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {
        in.defaultReadObject(); // Default deserialization

        // Custom deserialization (e.g., decrypt)
        String encryptedData = (String) in.readObject();
        this.sensitiveData = decrypt(encryptedData);
    }

    // Encryption/decryption methods
    private String encrypt(String data) {
        // Encryption logic
        return "ENCRYPTED:" + data;
    }

    private String decrypt(String encryptedData) {
        // Decryption logic
        return encryptedData.substring("ENCRYPTED:".length());
    }
}

```

```
}  
}
```

12. Multithreading

Creating Threads:

java



// Extending Thread class

```
class MyThread extends Thread {  
    @Override  
    public void run() {  
        System.out.println("Thread running: " + Thread.currentThread().getName());  
    }  
}
```

// Implementing Runnable interface

```
class MyRunnable implements Runnable {  
    @Override  
    public void run() {  
        System.out.println("Runnable running: " + Thread.currentThread().getName());  
    }  
}
```

// Using the Thread class

```
MyThread thread1 = new MyThread();  
thread1.start(); // Creates and runs a new thread
```

// Using Runnable

```
Thread thread2 = new Thread(new MyRunnable());  
thread2.start();
```

// Using lambda (Java 8+)

```
Thread thread3 = new Thread(() -> {  
    System.out.println("Lambda thread: " + Thread.currentThread().getName());  
});  
thread3.start();
```



Thread States and Lifecycle:

- NEW: Thread created but not started
- RUNNABLE: Thread executing or ready to execute
- BLOCKED: Thread blocked waiting for a monitor lock
- WAITING: Thread waiting indefinitely for another thread
- TIMED_WAITING: Thread waiting for a specified time
- TERMINATED: Thread completed execution

java



```
Thread thread = new Thread(() -> {  
    // Thread code  
});  
  
// Get thread state  
Thread.State state = thread.getState(); // NEW  
  
// Thread methods  
thread.start(); // Start thread execution  
thread.setName("Worker Thread"); // Set thread name  
thread.setPriority(Thread.MAX_PRIORITY); // Set priority (1-10)  
thread.join(); // Wait for thread to die  
thread.join(1000); // Wait up to 1000ms for thread to die  
thread.interrupt(); // Interrupt thread  
  
// Static Thread methods  
Thread current = Thread.currentThread(); // Get current thread  
Thread.sleep(1000); // Sleep for 1000ms (throws InterruptedException)  
Thread.yield(); // Yield to other threads
```

Thread Synchronization:

java



```
// Synchronized method
public synchronized void incrementCounter() {
    counter++;
}

// Synchronized block
public void incrementCounter() {
    synchronized(this) {
        counter++;
    }
}

// Static synchronized method
public static synchronized void incrementStaticCounter() {
    staticCounter++;
}

// Static synchronized block
public static void incrementStaticCounter() {
    synchronized(MyClass.class) {
        staticCounter++;
    }
}
```

Object-Level Locks and Wait/Notify:

java



```
class MessageQueue {
    private String message;
    private boolean empty = true;

    // Producer method
    public synchronized void put(String message) throws InterruptedException {
        while (!empty) {
            // Wait until queue is empty
            wait();
        }

        // Add message to queue
        this.message = message;
        empty = false;

        // Notify consumer
        notify();
    }

    // Consumer method
    public synchronized String take() throws InterruptedException {
        while (empty) {
            // Wait until message is available
            wait();
        }

        // Remove message from queue
        empty = true;

        // Notify producer
        notify();
    }
}
```

```
        return message;  
    }  
}
```

Lock Interface and ReentrantLock:

java



```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class BankAccount {
    private double balance;
    private final Lock lock = new ReentrantLock();

    public void deposit(double amount) {
        lock.lock();
        try {
            balance += amount;
        } finally {
            lock.unlock(); // Always unlock in finally block
        }
    }

    public boolean withdraw(double amount) {
        lock.lock();
        try {
            if (balance >= amount) {
                balance -= amount;
                return true;
            }
            return false;
        } finally {
            lock.unlock();
        }
    }
}
```

ReadWriteLock:

java



```
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;

class SharedResource {
    private final ReadWriteLock lock = new ReentrantReadWriteLock();
    private String data = "Initial data";

    public String read() {
        lock.readLock().lock();
        try {
            return data;
        } finally {
            lock.readLock().unlock();
        }
    }

    public void write(String newData) {
        lock.writeLock().lock();
        try {
            data = newData;
        } finally {
            lock.writeLock().unlock();
        }
    }
}
```

Thread-Safe Collections:

java



```
import java.util.concurrent.*;
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import java.util.ArrayList;
import java.util.List;

// Synchronized collections
List<String> synchronizedList = Collections.synchronizedList(new ArrayList<>());
Map<String, Integer> synchronizedMap = Collections.synchronizedMap(new HashMap<>());

// Concurrent collections
ConcurrentMap<String, Integer> concurrentMap = new ConcurrentHashMap<>();
BlockingQueue<String> blockingQueue = new LinkedBlockingQueue<>();
CopyOnWriteArrayList<String> copyOnWriteList = new CopyOnWriteArrayList<>();
```



Executor Framework:

java



```
import java.util.concurrent.*;

// Single thread executor
ExecutorService singleExecutor = Executors.newSingleThreadExecutor();

// Fixed thread pool
ExecutorService fixedPool = Executors.newFixedThreadPool(5);

// Cached thread pool
ExecutorService cachedPool = Executors.newCachedThreadPool();

// Scheduled thread pool
ScheduledExecutorService scheduledPool = Executors.newScheduledThreadPool(3);

// Submitting tasks
Future<?> future1 = fixedPool.submit(() -> {
    System.out.println("Task executing");
});

// Submitting tasks with results
Future<String> future2 = fixedPool.submit(() -> {
    return "Task result";
});
String result = future2.get(); // Blocks until result is available

// Scheduling tasks
scheduledPool.schedule(() -> {
    System.out.println("Delayed task");
}, 1, TimeUnit.SECONDS);

scheduledPool.scheduleAtFixedRate(() -> {
    System.out.println("Repeating task");
});
```

```
}, 0, 1, TimeUnit.SECONDS);
```

```
// Shutting down  
fixedPool.shutdown();
```

CompletableFuture (Java 8+):

java



```
import java.util.concurrent.CompletableFuture;

// Creating CompletableFuture
CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
    // Long running task
    return "Result";
});

// Chaining operations
CompletableFuture<String> finalFuture = future
    .thenApply(result -> result + " processed") // Transform result
    .thenCompose(result -> getAnotherFuture(result)) // Chain another CompletableFuture
    .thenCombine(CompletableFuture.supplyAsync(() -> "Combined"),
        (result1, result2) -> result1 + " " + result2) // Combine with another CompletableFuture
    .exceptionally(ex -> "Error: " + ex.getMessage()); // Handle exceptions

// Callback when complete
future.thenAccept(result -> System.out.println("Got result: " + result));

// Join multiple futures
CompletableFuture<String> future1 = CompletableFuture.supplyAsync(() -> "First");
CompletableFuture<String> future2 = CompletableFuture.supplyAsync(() -> "Second");

CompletableFuture<Void> allFutures = CompletableFuture.allOf(future1, future2);
allFutures.thenRun(() -> {
    // This runs when both futures complete
    String result1 = future1.join();
    String result2 = future2.join();
    System.out.println(result1 + " " + result2);
});
```



Thread Local Variables:

java



```
// Thread-local variable
private static ThreadLocal<Integer> threadId = ThreadLocal.withInitial(() -> 0);

// In thread code
void threadMethod() {
    int myId = threadId.get();
    threadId.set(myId + 1);

    // Do thread-specific work

    // Clean up
    threadId.remove();
}
```

13. JDBC (Java Database Connectivity)

Database Connection:

java



```
import java.sql.*;

// Loading the JDBC driver
Class.forName("com.mysql.cj.jdbc.Driver");

// Establishing connection
String url = "jdbc:mysql://localhost:3306/mydatabase";
String username = "root";
String password = "password";

try (Connection conn = DriverManager.getConnection(url, username, password)) {
    System.out.println("Database connected!");

    // Connection pooling with a better practice (using a connection pool)
    // HikariCP, Apache DBCP, or application server's connection pool

} catch (SQLException e) {
    e.printStackTrace();
}
```

Basic CRUD Operations:

java



// CREATE - Insert data

```
try (Connection conn = DriverManager.getConnection(url, username, password);
     PreparedStatement pstmt = conn.prepareStatement(
         "INSERT INTO users (name, email, age) VALUES (?, ?, ?)")) {

    pstmt.setString(1, "John Doe");
    pstmt.setString(2, "john@example.com");
    pstmt.setInt(3, 30);

    int rowsAffected = pstmt.executeUpdate();
    System.out.println(rowsAffected + " row(s) inserted");
}
```

// READ - Query data

```
try (Connection conn = DriverManager.getConnection(url, username, password);
     PreparedStatement pstmt = conn.prepareStatement("SELECT * FROM users WHERE age > 18")) {

    pstmt.setInt(1, 18);

    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            int id = rs.getInt("id");
            String name = rs.getString("name");
            String email = rs.getString("email");
            int age = rs.getInt("age");

            System.out.printf("ID: %d, Name: %s, Email: %s, Age: %d\n",
                              id, name, email, age);
        }
    }
}
```

```

// UPDATE - Update data
try (Connection conn = DriverManager.getConnection(url, username, password);
     PreparedStatement pstmt = conn.prepareStatement(
         "UPDATE users SET age = ? WHERE id = ?")) {

    pstmt.setInt(1, 31);
    pstmt.setInt(2, 1);

    int rowsAffected = pstmt.executeUpdate();
    System.out.println(rowsAffected + " row(s) updated");
}

// DELETE - Delete data
try (Connection conn = DriverManager.getConnection(url, username, password);
     PreparedStatement pstmt = conn.prepareStatement(
         "DELETE FROM users WHERE id = ?")) {

    pstmt.setInt(1, 1);

    int rowsAffected = pstmt.executeUpdate();
    System.out.println(rowsAffected + " row(s) deleted");
}

```

Transactions:

java



```
Connection conn = null;
try {
    conn = DriverManager.getConnection(url, username, password);

    // Disable auto-commit
    conn.setAutoCommit(false);

    // First statement
    try (PreparedStatement pstmt1 = conn.prepareStatement(
        "UPDATE accounts SET balance = balance - ? WHERE id = ?")) {
        pstmt1.setDouble(1, 100.0);
        pstmt1.setInt(2, 1);
        pstmt1.executeUpdate();
    }

    // Second statement
    try (PreparedStatement pstmt2 = conn.prepareStatement(
        "UPDATE accounts SET balance = balance + ? WHERE id = ?")) {
        pstmt2.setDouble(1, 100.0);
        pstmt2.setInt(2, 2);
        pstmt2.executeUpdate();
    }

    // If everything is OK, commit
    conn.commit();
    System.out.println("Transaction committed");

} catch (SQLException e) {
    // If there's an error, rollback
    if (conn != null) {
        try {
            conn.rollback();
        }
    }
}
```

```

        System.out.println("Transaction rolled back");
    } catch (SQLException ex) {
        ex.printStackTrace();
    }
}
e.printStackTrace();
} finally {
    // Reset auto-commit mode and close connection
    if (conn != null) {
        try {
            conn.setAutoCommit(true);
            conn.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
}

```

Batch Processing:



```
try (Connection conn = DriverManager.getConnection(url, username, password);
     PreparedStatement pstmt = conn.prepareStatement(
         "INSERT INTO users (name, email, age) VALUES (?, ?, ?)")) {

    // Add multiple rows to batch
    for (int i = 0; i < 100; i++) {
        pstmt.setString(1, "User" + i);
        pstmt.setString(2, "user" + i + "@example.com");
        pstmt.setInt(3, 20 + i % 40);
        pstmt.addBatch();
    }

    // Execute batch
    int[] updateCounts = pstmt.executeBatch();
    System.out.println("Batch executed with " + updateCounts.length + " operation:");
}
```



Stored Procedures:

java



```
// Calling stored procedure with IN parameter
try (Connection conn = DriverManager.getConnection(url, username, password);
     CallableStatement cs = conn.prepareCall("{call get_user_by_id(?)}")) {

    cs.setInt(1, 1); // Set IN parameter

    try (ResultSet rs = cs.executeQuery()) {
        while (rs.next()) {
            System.out.println("User: " + rs.getString("name"));
        }
    }
}
```